AUTOMATION CASE STUDY

Maintenance request triage

A local n8n demonstration that turns unstructured tenant messages into manager-controlled maintenance decisions.

THE PROBLEM

Maintenance requests arrive through email, forms and messages with no consistent priority, trade or property context. A manager must read, sort and follow up on each one - often after hours.

THE BUILD

Five connected n8n workflows normalize intake, classify and store tickets, continue tenant clarification, prepare a manager decision, and run operational checks. Postgres owns durable state; Airtable is the manager view.

HUMAN APPROVAL IS THE SEND GATE

Vendor email is sent only after a manager selects a vendor in Telegram. A Hold or review-only path sends no vendor email. The tenant hears that a provider was contacted only after SMTP succeeds.

SYSTEM DESIGN

One connected decision flow

The workflow exports, Postgres schema and local n8n editor show five published operational workflows. Historical exports remain in the repository but are not part of this live set.

01 **Tenant Intake**

IMAP / form -> normalized request -> Triage Engine -> pairing link

02 **Triage Engine**

Deduplicate -> strict model output -> safety rules -> Postgres ticket / draft

03 **Tenant Telegram**

Pairing, direct issue intake, clarification and factual status replies

04 **Manager Desk**

Vendor match -> Telegram choice -> one claimed SMTP send -> tenant update

05 **Operations**

Digest, heartbeat, unanswered emergencies and Airtable conversation sync

State and handoffs: Postgres is authoritative for tickets, messages, callbacks and one-send claims. Airtable mirrors requests and conversations and supplies demo vendor/tenant rosters. OpenAI output remains advisory.

Two sources, one triage path

Email IMAP and the n8n form normalize into one message contract and invoke the Triage Engine.



Actual n8n editor capture - complete 15-node workflow

- The intake context is preserved before the sub-workflow call so source identifiers and reply details survive database lookups.
- A stored ticket can trigger a one-time Telegram pairing email; the system subject prefix prevents shared-mailbox loops.

Classify, check, persist

The reusable core takes normalized text and returns a stored outcome or an existing-ticket/thread result.

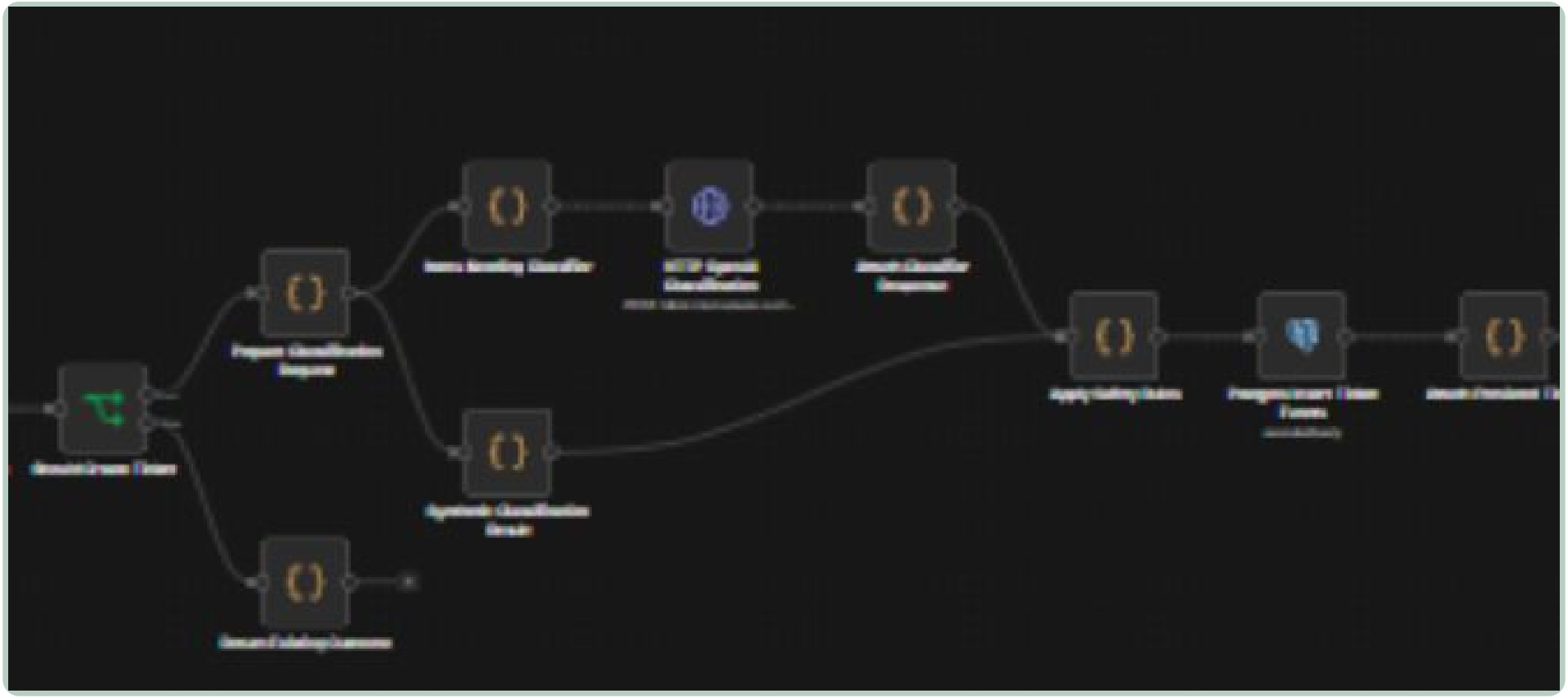


Actual n8n editor capture - complete 25-node workflow

- It deduplicates and threads before calling the Responses API through HTTP Request. The installed n8n node did not expose the required strict response contract.
- Fixed safety rules can raise urgency; model failure and sensitive requests route to human review.

The classifier and safety boundary

This enlarged editor section shows the model branch joining the deterministic safety step.

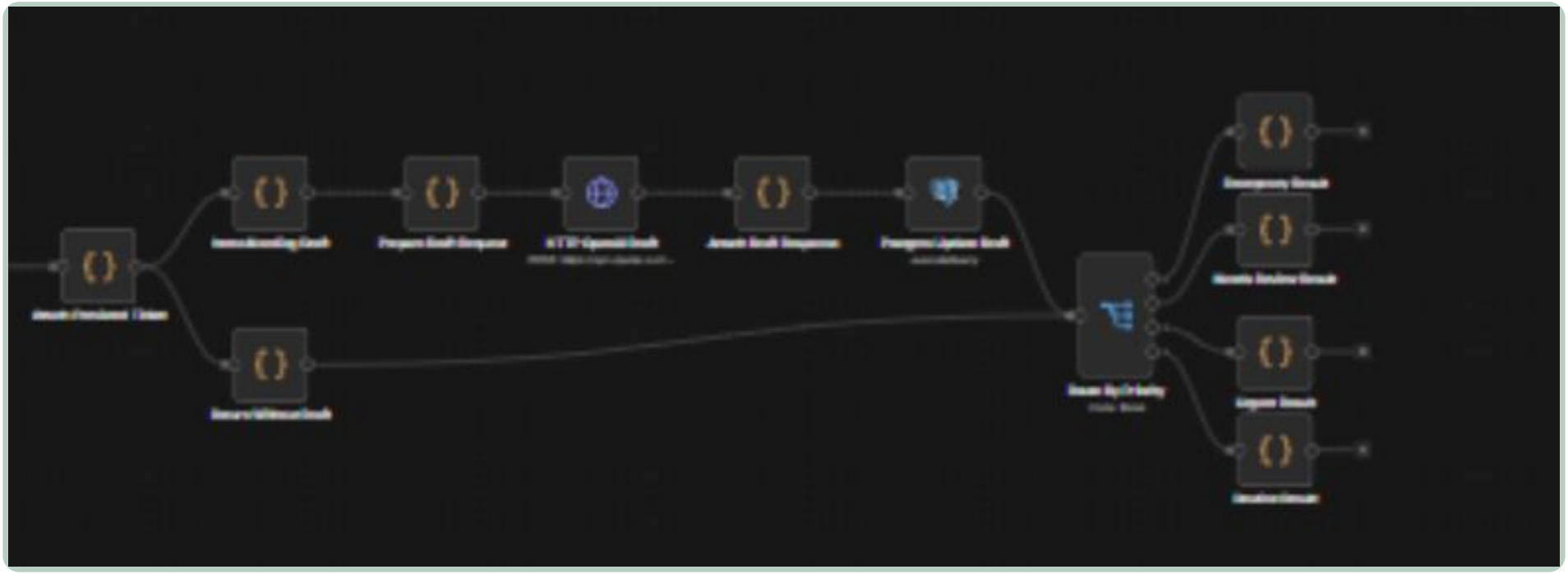


Actual editor crop - classification, safety, persistence

The model supplies structured priority, trade, missing information and rationale. Post-model rules handle known safety and maintenance edge cases. Postgres records the result and audit events before any draft path continues.

Draft only when allowed

The draft path branches from the persisted ticket and rejoins priority routing.

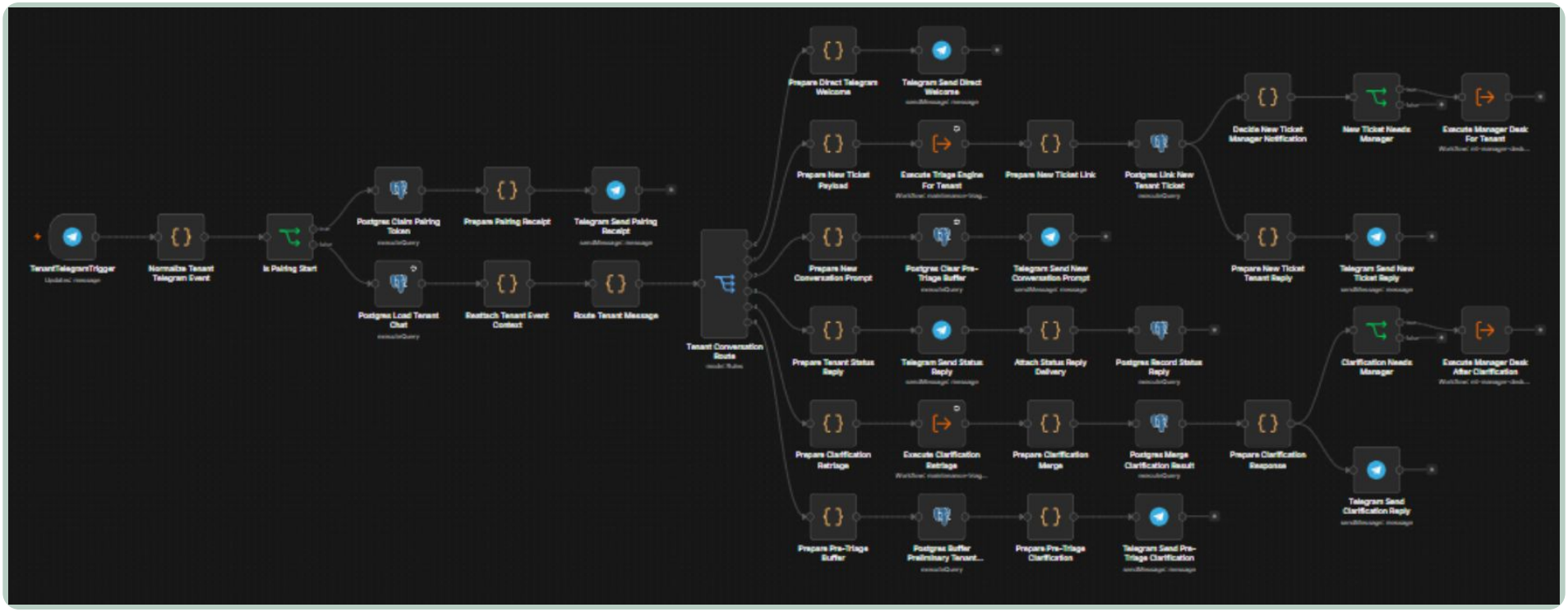


Actual editor crop - draft and output routing

Review-only and human-only requests bypass drafting. Eligible maintenance requests get a structured vendor draft; the workflow then routes emergency, urgent, routine and review outputs to downstream handling.

A bounded tenant conversation

The dedicated demo tenant bot handles pairing, direct requests, missing details, new-issue commands and status questions.

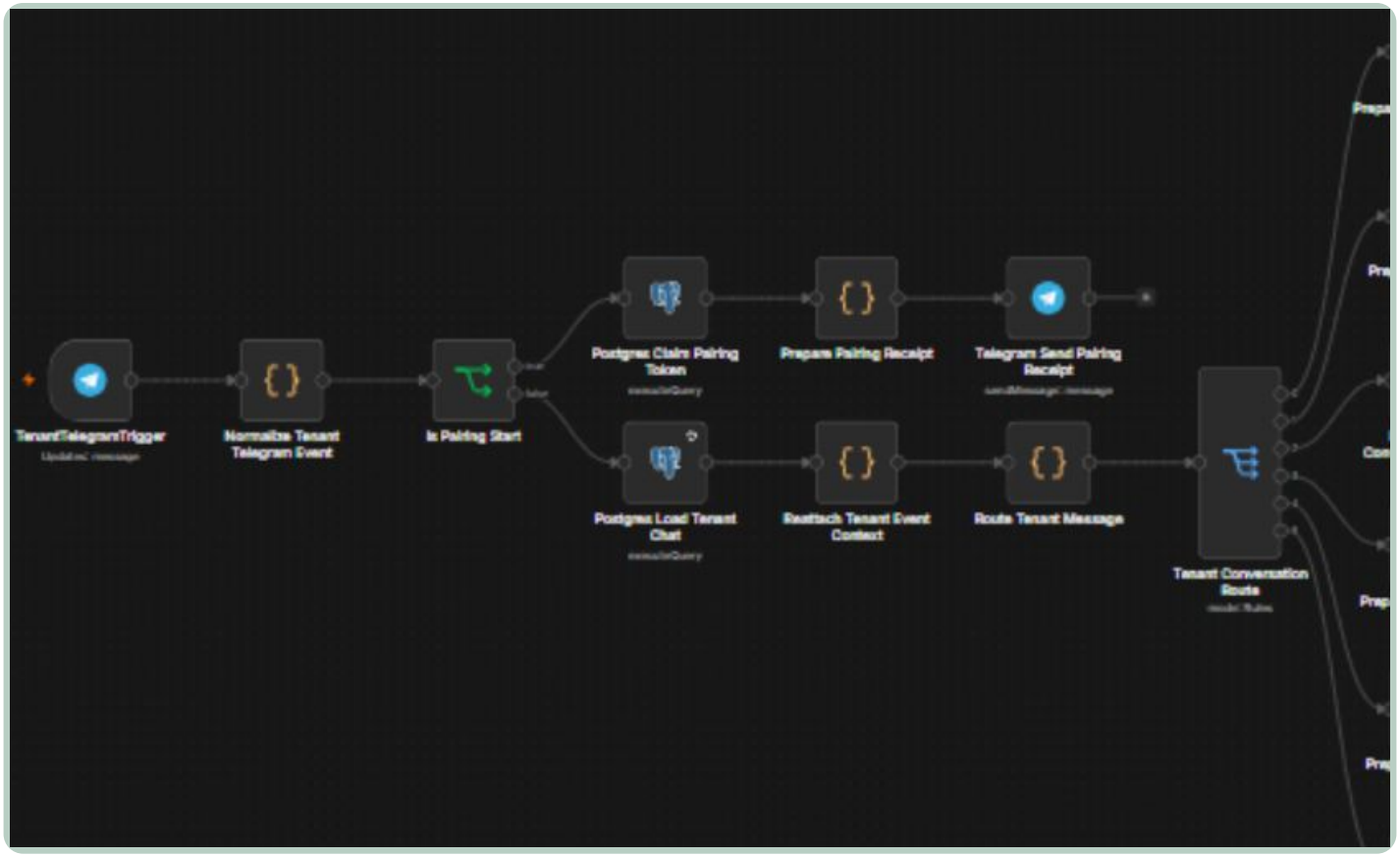


Actual n8n editor capture - complete 40-node workflow

- A generic opening is buffered until the tenant names a concrete symptom, so it creates no ticket or manager alert.
- Status intent is handled before new-ticket creation; clarification reuses the same request and calls the manager only once reviewable.

Pair or route the message

This enlarged section shows the entry gate and conversation router.

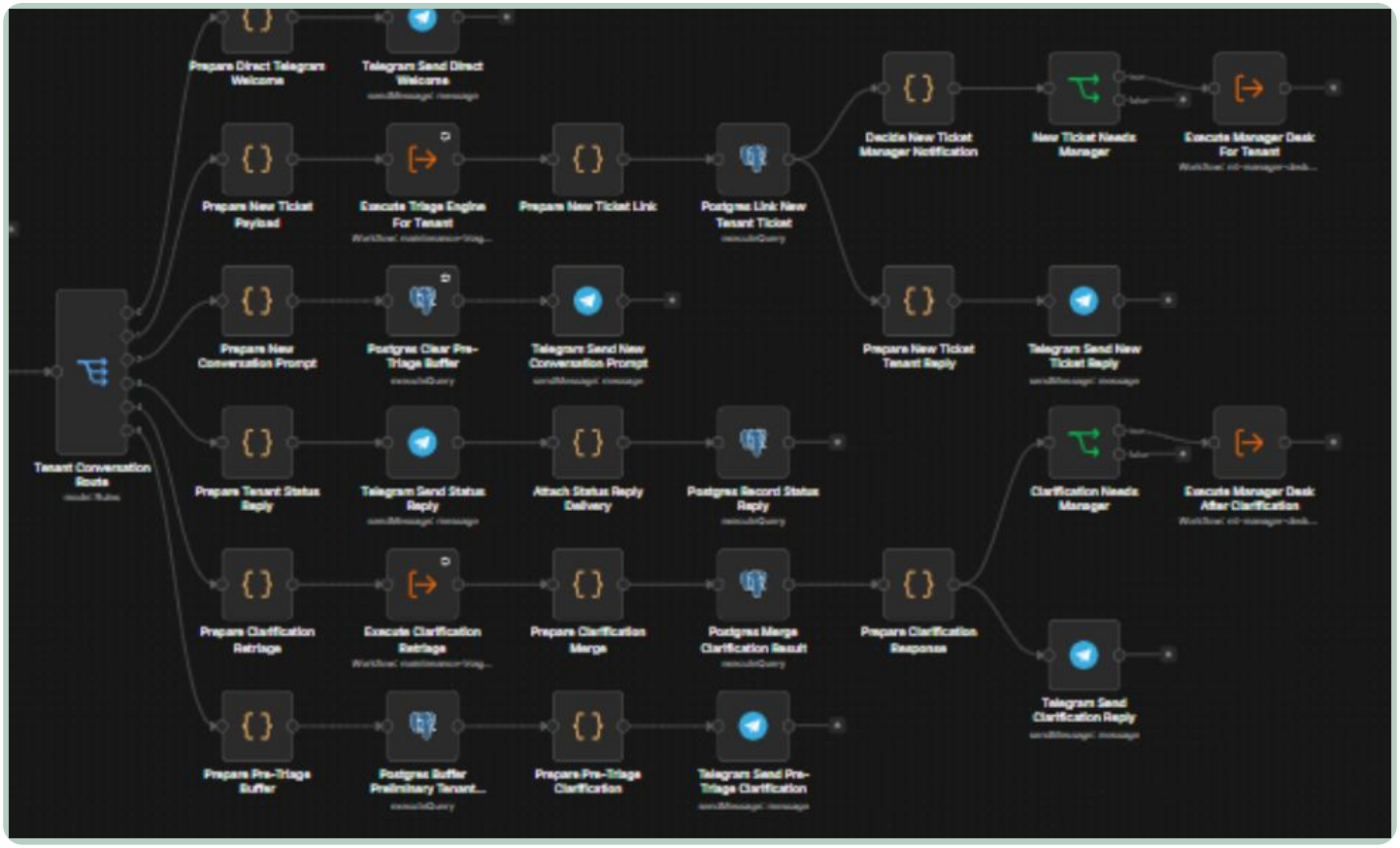


Actual editor crop - pairing and message routing

The bot loads the chat and reattaches trigger context after Postgres lookup. Pairing claims a one-time token. Other messages are separated into welcome, new ticket, preliminary buffer, status and clarification paths.

Keep one issue in one thread

This enlarged section shows ticket creation, status reply and clarification routes.

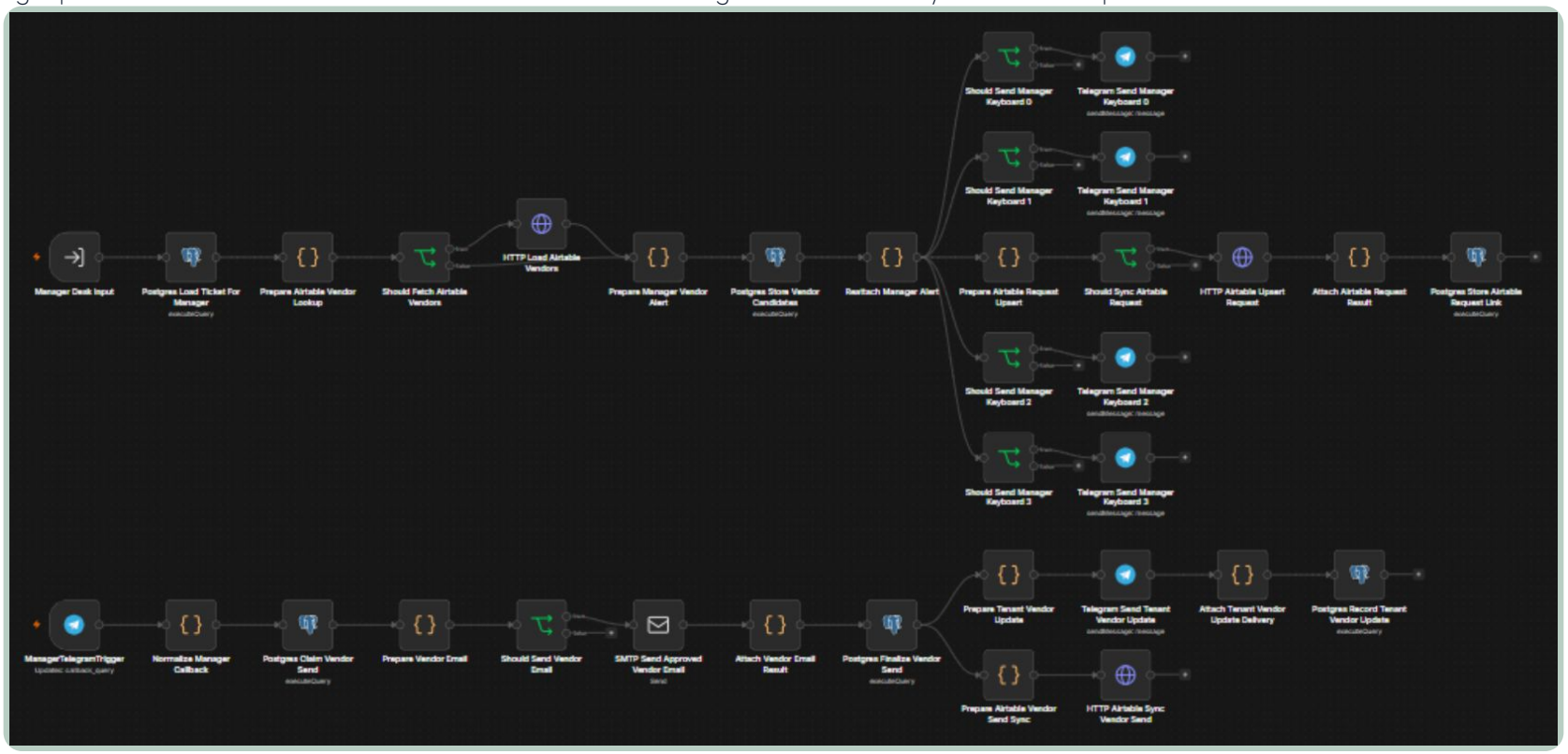


Actual editor crop - new ticket, status and clarification

A concrete issue calls the Triage Engine and links the ticket to the chat. Missing fields prompt the tenant; a status question records one factual reply; a completed issue can invoke the Manager Desk.

Approval before vendor contact

The manager path matches candidates from Airtable and sends a Telegram alert with only valid choices plus Hold.

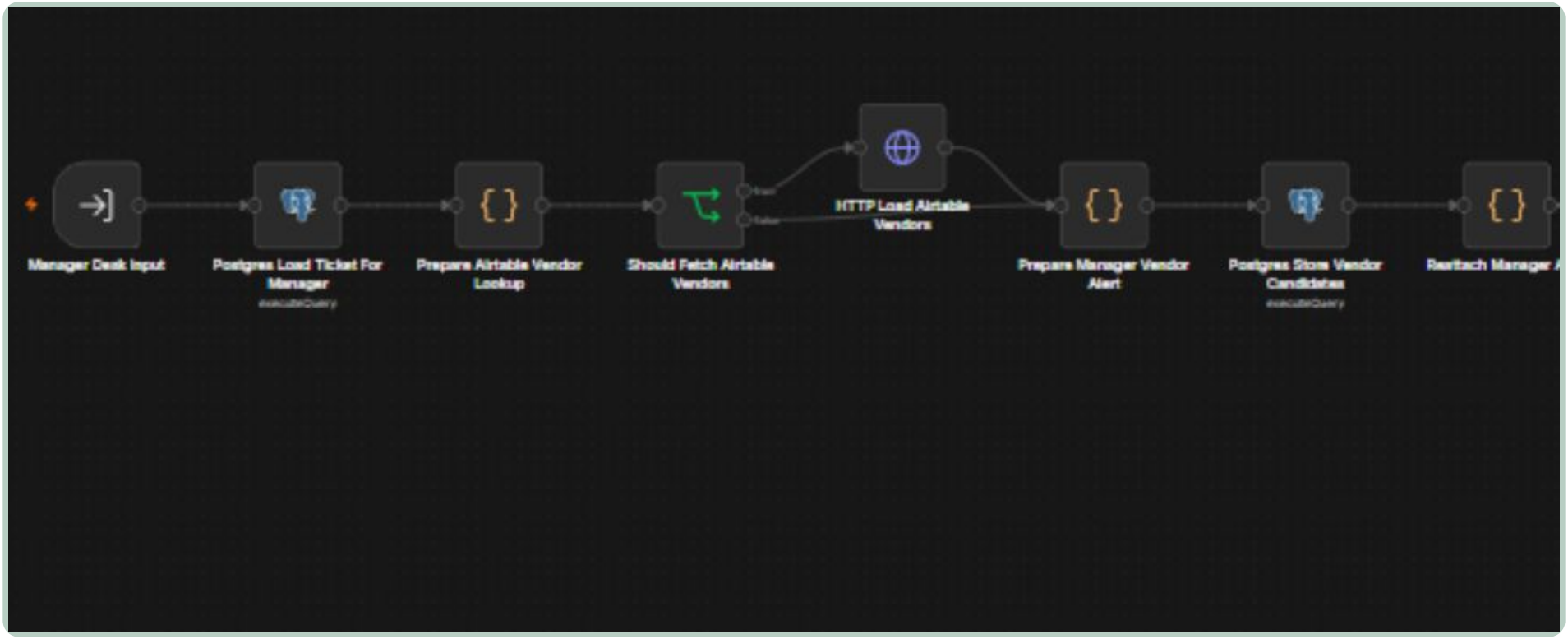


Actual n8n editor capture - complete 35-node workflow

- Review-only, incomplete and no-match tickets cannot expose a vendor-send button.
- The callback claims a unique Postgres send attempt before SMTP; successful delivery is recorded before any tenant provider-contacted update.

Candidate and alert path

The upper route loads the ticket, finds matching vendors and assembles the manager choice.



Actual editor crop - ticket lookup, vendor matching, alert preparation

Candidates match property and trade and are ordered by configured vendor priority. The request and conversation state are mirrored into Airtable, while Postgres retains the durable approval state.

One approved SMTP send

The lower route is activated by a manager Telegram callback.



Actual editor crops - callback claim, SMTP and recorded outcomes

The unique vendor-send claim prevents a second tap from resending. On SMTP success, the result is persisted before the tenant update. A failed send leaves the tenant uninformed about provider contact.



Actual n8n editor capture - complete 17-node workflow

- Daily digest and operator heartbeat are implemented but config-gated off in the clean demo state.
- Unanswered emergencies can re-alert the manager; conversation sync appends stored messages to Airtable. Age alone creates overdue visibility, not an emergency phone alert.

What the demo actually proves

All results below come from synthetic fixtures, controlled mailboxes or local workflow checks - not a client deployment.

VERIFIED

- Stage 8 five-run rehearsal: 30/30 expected priorities across six fixtures x five clean runs; zero fixture-mode Telegram sends.
- Stage 9 conversation-routing regression and structural validation passed; live webhook status question created no extra ticket or manager action.
- Controlled live tests included form ingestion, Airtable request sync, Telegram manager alerts and an approved SMTP send.
- Six-fixture historical nano scorecard: 6/6 priority matches, zero under/over-escalations and zero missed safety signals; average call-plus-draft latency 10.9 s.

LIMITATIONS + MANUAL STEPS

- The full 30-fixture x three-run nano hardening gate failed with under-escalations and missed safety signals. The live workflow now uses gpt-5.6-luna; broad accuracy is unproven.
- Telegram callbacks need a current HTTPS tunnel and operator-controlled bot/chat setup. Final tenant-originated live checks remain operator actions.
- The demo bot represents one known tenant; pairing links do not verify identity. No PMS integration, appointment scheduling, automatic closure or production hosting is included.